

Homework 1

Neighborhood Sports Preferences Simulation

COMS 2280 Fall 2025

Due at **11:59 PM, Sept. 26th**

Contents

1	Project Overview	2
2	Core Concepts	2
2.1	Neighborhood Grid Example	2
2.2	Simulation Flow:	2
3	Assignment Tasks	3
3.1	Importing the skeletal code	3
3.1.1	Importing in Eclipse	3
3.1.2	Importing in IntelliJ	3
3.2	To Do	3
4	Class Hierarchy Overview	4
5	Simulation Rules	4
5.1	Baseball (A)	4
5.2	Basketball (B)	4
5.3	Everything (E)	4
5.4	Football (F)	5
5.5	Nothing (N)	5
5.6	Rugby (R)	5
5.7	Soccer (S)	5
6	Input/Output Format	5
7	Class Description	6
7.1	Neighborhood Class	6
7.2	NeighborhoodGrid Class	6
7.3	Household Abstract Class	7
7.4	SportsHouseholds Abstract Class	7
7.5	InterestLevel Interface	7
7.6	Enum: Sports	7
7.7	Household Types and Rules	7
8	Testing	8
9	Submission	8
9.1	Deliverables	8
9.2	Important Instructions	8
9.3	Significant Dates	9
10	Working Example	9

1 Project Overview

The “Neighborhood Sports Preferences” simulation project models the dynamic nature of sports interests within a neighborhood. It simulates a neighborhood represented by a square grid of households, each with a particular sport preference (or no preference at all). The simulation progresses in monthly steps, where each household’s preference can change based on the preferences of its neighbors.

2 Core Concepts

The simulation operates on a square grid of households. Each household has a “preference” for a particular sport and, for some sports, an “interest level” in that sport. The core of the simulation is an update loop that advances month by month. In each month, every household in the grid determines its state for the *next* month based on a set of rules. These rules are primarily influenced by the sports preferences of the household’s immediate neighbors.

At any point in the simulation, the grid contains households with one of the following sports preferences:

1. **Baseball** (A)
2. **Basketball** (B)
3. **Football** (F)
4. **Rugby** (R)
5. **Soccer** (S)
6. **Everything** (E): A general interest in all sports.
7. **Nothing** (N): No interest in any sports.

Households with a specific sports preference (like Baseball, Basketball, etc.) also have an **interest level**, which represents their interest for that sport.

The **neighbors** of a household consist of the 3×3 grid of cells centered on that household. For households on the edge of the grid, the neighborhood size can be smaller.

2.1 Neighborhood Grid Example

Below is a neighborhood grid of size 5.

```
0  N   F0  N   B3  B4
1  B3  B1  A0  E   E
2  N   E   N   A4  E
3  N   B0  A4  E   N
4  E   E   A0  A3  F2
```

The grid indices start at 0, and each square is identified by its (row, column) indices. In this example the square (1,1) is occupied by a household that is interested in basketball with an interest level of 1 (B1). Its neighbors are:

```
N   F0  N
B3  B1  A0
N   E   N
```

The grid (0,0) is occupied by a household that shows no interest in sports (N). Its neighbors are:

```
N   F0
B3  B1
```

Meanwhile, the square (2,0) is also occupied by a household with no interest in sports (N) and its neighbors are:

```
B3  B1
N   E
N   B0
```

2.2 Simulation Flow:

1. **Initialization:** The simulation starts with a grid of households. This grid can be initialized in two ways:
 - (a) **Randomly:** A grid of user-specified size is created with each cell being assigned a random household type and, if applicable the interest level of each household in the grid.
 - (b) **From a file:** A text file specifies the type and, if applicable, the interest level of each household in the grid.
2. **Update Loop:** The simulation proceeds in discrete monthly steps (**starting at 0**) for a user-specified number of months. In each step, a new grid is created, and each cell of this new grid is populated based on the `next()` state of the household in the corresponding cell of the `old` grid.
3. **Neighborhood Survey:** The decision-making process for each household’s next state begins with a “survey” of its eight immediate neighbors (and itself), i.e., the survey occurs over a 3×3 grid. When the household is on

the border of the grid, the survey may be conducted over 2×3 , 3×2 or 2×2 grid. The simulation counts the number of neighbors with each type of sports preference.

4. **Rule Application:** Based on the survey results, each household applies a specific set of rules to determine its type and interest level for the next month.
5. **Termination:** After the specified number of months, the simulation displays the final state of the neighborhood grid.

3 Assignment Tasks

3.1 Importing the skeletal code

You are given a skeletal code containing the following class files:

1. `Household.java`
2. `InterestLevel.java`
3. `Neighborhood.java`
4. `NeighborhoodGrid.java`
5. `Sports.java`
6. `SportsHouseholds.java`

Import this project into your IDE of choice. This will enable you to quickly get started as well make sure that JUnit dependency is pulled in for writing test cases.

3.1.1 Importing in Eclipse

1. In Eclipse, go to the menu bar and select **File > Import...**
2. In the import window that appears, expand the Maven folder and select **Existing Maven Projects**. Click **Next**.
3. Choose the Project Directory:
 - Click the **Browse...** button next to the **Root Directory** field.
 - Navigate to and select the root folder of your Maven project. This is the folder that contains the `pom.xml` file, not the `src/main/java` folder itself.
4. Eclipse will automatically scan the folder and its subdirectories for `pom.xml` files.
5. Once the scan is complete, your project (identified by its `pom.xml`) will appear in the **Projects** box. Make sure the checkbox next to it is ticked.
6. Click the **Finish** button and you are ready to go.

3.1.2 Importing in IntelliJ

1. In IntelliJ, go to the menu bar and select **File > Open...**
2. A file navigator window will appear. You have two primary options:
 - **Best Option:** Navigate to your project's root folder and select the `pom.xml` file directly.
 - **Alternative:** Select the entire root folder of your project (the one containing the `pom.xml` file).
3. **Confirm and Open:** Click **OK**.
4. IntelliJ will recognize the `pom.xml` and ask how to open it. Choose **Open as Project**.
5. A dialog will pop up asking if you **Trust and Open** the Maven project. Click **Trust Project** to allow IntelliJ to build the project and download its dependencies.

3.2 To Do

- Look through the files and implement any incomplete functions. For example,
 - `survey` in `Household`.
 - `updateGrid` in `Neighborhood`.
 - Constructors for initializing `NeighborhoodGrid`.
- Implement concrete classes for households representing `Everything` and `Nothing` by extending `Household`.
- Implement concrete classes for `Baseball`, `Basketball`, `Football`, `Rugby`, and `Soccer` by extending `SportsHouseholds`.
 - Important functions are `getPreference`, `next`, `toString` and constructors.

Check the class hierarchy given in Figure 1 and the class descriptions in Section 4 and 7 to find out the method signatures and their usages. Use the rules outlined in Section 5 for implementing the `next` methods.

4 Class Hierarchy Overview

We use a class hierarchy to model the simulation. The abstract `Household` class serves as the base for all household types and contains methods for surveying neighbors and determining the next state. It is extended by the abstract `SportsHouseholds` class, which handles the `interestLevel` for households with specific sports preferences. The `InterestLevel` interface is used to define the `getInterest()` method.

The concrete household types (`Baseball`, `Basketball`, `Football`, `Rugby`, `Soccer`, `Everything`, `Nothing`) implement the specific evolution rules in their `next()` methods.

A `NeighborhoodGrid` class represents the grid, storing a 2D array of `Household` objects. It includes methods for, initializing a grid from file or randomly, retrieving the grid size as well as outputting the grid configuration.

The `Neighborhood` class contains the main logic for running the simulation. It handles user input to select a grid source (random or file), the number of months to simulate, and then runs the simulation loop, printing the initial, intermediate, and final grids. It also includes an `updateGrid()` method to calculate the state of the grid for the next month.

A class hierarchy diagram is shown in Figure 1.

5 Simulation Rules

The sports preference of a household in the next month is determined by the preferences of its neighbors in the current month, according to a set of rules specific to each household type.

5.1 Baseball (A)

A Baseball household's interest level increases by one each month, unless one of the following prioritized rules is met:

1. If its interest level reaches the maximum, it changes to a Nothing household.
2. If there are more than 3 neighboring Soccer households, it changes to a Soccer household with an interest level of 0.
3. If there are fewer than 2 neighboring Baseball households, it changes to a Rugby household with an interest level of 0.
4. If the combined number of neighboring Baseball and Soccer households is greater than 5, it changes to a Nothing household.
5. If the number of neighboring Football households is more than twice the number of neighboring Baseball households, it changes to a Football household with an interest level of 0.

5.2 Basketball (B)

A Basketball household's interest level increases by one each month, unless one of the following prioritized rules is met:

1. If its interest level reaches the maximum, it changes to a Nothing household.
2. If there are more than 5 neighboring Football households, it changes to a Football household with an interest level of 2.
3. If there are at least 2 neighboring Soccer households, it changes to a Soccer household with an interest level of 0.
4. If there are fewer than 2 neighboring Basketball households, it changes to Everything household.
5. If the combined number of neighboring Baseball, Basketball, and Football households is greater than 6, it changes to Everything household.

5.3 Everything (E)

A household with a preference for Everything will change its preference based on the following prioritized rules:

1. If the number of neighboring households with a Soccer preference is at least three times the number of neighbors with Everything preference, it changes to a Soccer household with an interest level of 3.
2. If there are more than 3 neighboring Football households, it changes to a Football household with an interest level of 0.
3. If the number of neighboring households with no preference (Nothing) is greater than combined number of Football and Soccer households, it changes to a Basketball household with an interest level of 2.
4. If the number of neighboring Rugby households is greater than the number of neighbors with Everything preference, it changes to a Rugby household with an interest level of 0.
5. Otherwise, it remains Everything household.

5.4 Football (F)

A Football household has special rules for months 4, 5, and 6 of the year (April, May, and June). During these months, if there is at least one neighboring Everything or Nothing household, its interest level will increase (not past the maximum interest level), but it will not change to another household type.

For all other months and situations, a Football household's interest level increases by one each month, unless one of the following prioritized rules is met:

1. If its interest level reaches the maximum, it changes to a Nothing household.
2. If the combined number of neighboring Baseball, Basketball, and Football households is greater than 7, it changes to Everything household.
3. If there are more than 3 neighboring Basketball households, it changes to a Basketball household with an interest level of 0.
4. If there are fewer than 2 neighboring Football households, it changes to a Baseball household with an interest level of 0.

5.5 Nothing (N)

A household with no sports preference will change its preference based on the following prioritized rules:

1. If there are more than 5 neighboring Soccer households, it changes to a Soccer household with an interest level of 5.
2. If there are more than 4 neighboring Football households, it changes to a Football household with an interest level of 0.
3. If there are more than 3 neighboring Basketball households, it changes to a Basketball household with an interest level of 0.
4. If there are more than 2 neighboring Baseball households, it changes to a Baseball household with an interest level of 0.
5. If there is more than 1 neighboring Rugby household, it changes to a Rugby household with an interest level of 2.
6. If there is at least one neighboring Everything household, it changes to an Everything household.
7. Otherwise, it remains a Nothing household.

5.6 Rugby (R)

A Rugby household's interest level increases by one each month, unless one of the following prioritized rules is met:

1. If its interest level reaches the maximum, it changes to a Nothing household.
2. If the combined number of neighboring Football and Soccer households is at least 8, it changes to a Soccer household with an interest level of 2.
3. If the number of neighboring Baseball households is more than twice the number of neighboring Basketball households, it changes to a Baseball household with an interest level of 4.
4. If there are fewer than 2 neighboring Rugby households, it changes to a Football household with an interest level of 0.

5.7 Soccer (S)

A Soccer household's interest level increases by one each month, unless one of the following prioritized rules is met:

1. If its interest level reaches the maximum, it changes to a Nothing household.
2. If the combined number of neighboring Football and Basketball households is greater than or equal to the number of neighboring Soccer households, it changes to Everything household.
3. If the number of neighboring Soccer households is greater than the combined number of neighboring Baseball, Basketball, Football, and Rugby households, it changes to a Soccer household with an interest level of 5.
4. If there are more than 4 neighboring Rugby households, it changes to a Rugby household with an interest level of 3.
5. Otherwise, if there are no neighboring Everything households, it changes to a Nothing household.

6 Input/Output Format

The format for representing the neighborhood grid for reading or outputting is as follows.

- Each household occupies three spaces in the grid.
- The first character is a letter representing the sports preference:

- **A** for Baseball
- **B** for Basketball
- **E** for Everything
- **F** for Football
- **N** for Nothing
- **R** for Rugby
- **S** for Soccer
- For households with an interest level, the letter is followed by a single digit representing the interest level (e.g., 0-5). A blank space follows the digit.
- For households with no interest level (Everything and Nothing), two blank space follows the letter.

Example: An output of “A1 ” represents a Baseball household with an interest level of 1. An output of “N ” represents a Nothing household. **Note the space following the output. This needs to be implemented in the toString().**

7 Class Description

The project is structured around several classes and an interface that define the components and behavior of the simulation.

7.1 Neighborhood Class

This is the main class that drives the simulation. It contains the `main` method, which serves as the entry point of the application.

- `main(String[] args):`
 - Presents a menu to the user to either generate a random grid, load a grid from a file, or exit.
 - Prompts the user for the number of months to simulate. The simulation should start with month 0.
 - Initializes the `NeighborhoodGrid`.
 - Runs the simulation loop, printing the state of the grid at each step.
 - Displays the final grid at the end of the simulation.
 - Refer to the working example given in Section 10 for the exact input / output format.
- `updateGrid(NeighborhoodGrid oldGrid, NeighborhoodGrid newGrid, int currentMonth):`
 - A static method that iterates through every household in the `oldGrid`.
 - For each household, it calls the `next()` method to determine the household’s state in the next month.
 - The resulting new household is placed in the corresponding position in the `newGrid`.

7.2 NeighborhoodGrid Class

This class represents the neighborhood itself, managing the 2D array of households.

- **Fields:**
 - `size`: An integer representing the width and height of the grid.
 - `grid`: A 2D array of `Household` objects.
- **Constructors:**
 - `NeighborhoodGrid(int size)`: Creates an empty grid of a specified size.
 - `NeighborhoodGrid(String inputFileName)`: Creates a grid by reading its configuration from a text file. The file format consists of space-separated strings, where each string represents a household. For example, “B3 ” would be a Basketball household with an interest level of 3, and “N ” would be a household with no interest in sports.
 - You can assume that the inputs given will be correct and don’t need to handle the exceptions.
- **Methods:**
 - `getSize()`: Returns the size of the grid.
 - `randomInit()`: Populates the grid with random household types. Use the `Random` class to make sure that all sports and interest levels are given an equal chance when initializing.
 - `toString()`: Returns a string representation of the grid, suitable for printing to the console.
 - `write(String outputFileName)`: Writes the current state of the grid to a file.

7.3 Household Abstract Class

This is the abstract base class for all types of households in the simulation. It defines the common properties and behaviors of a household.

- **Fields:**
 - `members`: A static integer array that is used to store the results of the neighborhood survey.
 - `MAX_INTEREST`: A static final integer, representing the maximum interest level a household can have.
 - `neighborhoodGrid`: A reference to the `NeighborhoodGrid` the household belongs to.
 - `row`, `column`: Integers representing the household's position in the grid.
- **Constructor:**
 - `Household(NeighborhoodGrid grid, int row, int column)`: Initializes the household's grid reference and position.
- **Methods:**
 - `survey(int households[])`: This method calculates the number of neighbors (including the household itself) for each sports preference. It populates the `households` array with these counts.
 - `getPreference()`: An abstract method that must be implemented by subclasses to return the `Sports` enum value corresponding to the household's preference.
 - `next(NeighborhoodGrid newGrid, int month)`: An abstract method that must be implemented by subclasses. This method contains the logic to determine the household's state in the next month and returns a new `Household` object representing that state.

7.4 SportsHouseholds Abstract Class

This abstract class extends `Household` and is the base class for all households that have an interest in a specific sport.

- **Field:**
 - `interestLevel`: An integer representing the household's interest in its preferred sport.
- **Constructor:**
 - `SportsHouseholds(NeighborhoodGrid grid, int row, int column, int interestLevel)`: Initializes the household, including its interest level.
- **Method:**
 - `getInterest()`: Returns the household's current interest level.

7.5 InterestLevel Interface

This interface defines a single method for retrieving the interest level of a household.

- **Method:**
 - `getInterest()`: Returns an integer representing the interest level.

7.6 Enum: Sports

This enumeration defines the possible sports preferences that a household can have.

- **Values:**
 - `BASEBALL`
 - `BASKETBALL`
 - `EVERYTHING`
 - `FOOTBALL`
 - `NOTHING`
 - `RUGBY`
 - `SOCCER`

7.7 Household Types and Rules

The simulation includes several concrete implementations of the `Household` class, each with its unique set of rules for determining its next state. All of them implements the following three functions.

- `getPreference()`: Retrieves the sports preference of the household.
- `toString()`: The string representation for the household in the grip.
- `next()`: Rules for updating the preference for next month as given in Section 5.

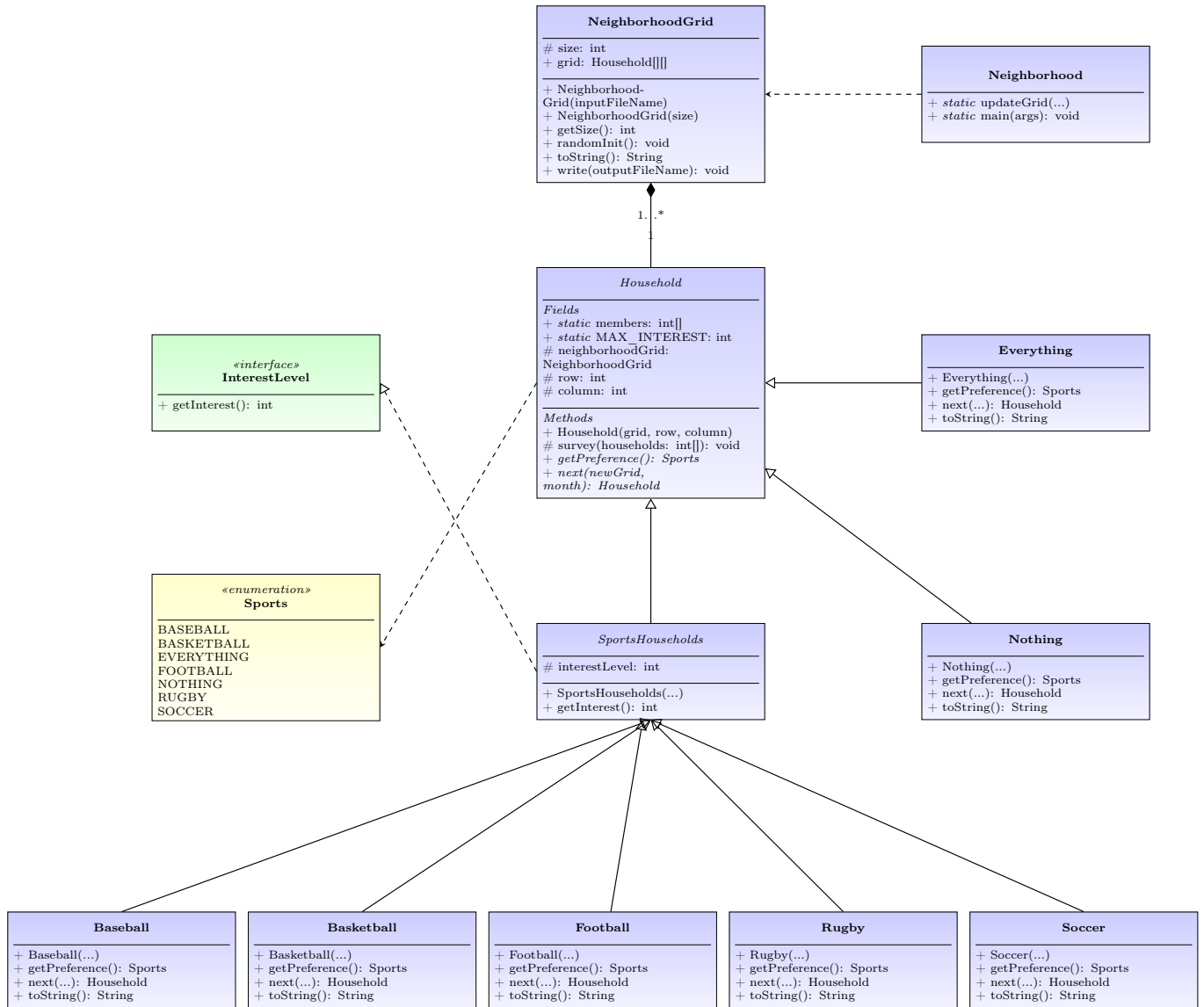


Figure 1: **Class Hierarchy:** UML Diagram for the Class Hierarchy

8 Testing

The project must include a comprehensive suite of unit tests using **JUnit 5**. Each household class should have a corresponding test class (e.g., `BaseballTest`, `SoccerTest`) that verifies the correctness of the `next()` method under various neighborhood configurations. There must also be tests for the `NeighborhoodGrid` and `Neighborhood` classes to ensure the overall simulation logic works as expected.

If you imported the project correctly as described in Section 3.1, necessary JUnit dependencies should already be available.

9 Submission

Write your classes and JUnit test classes in the `edu.iastate.cs2280.hw1` package.

9.1 Deliverables

- A zip file containing all the java source files. **Please don't turn in any generated class files.**
- The zip should include `src` folder and `pom.xml` at its root.
- If you have any files associated with your JUnit tests, include them.
- The zip file should be named **FirstName_LastName.zip**.
- Make sure the zip file you are uploading is the correct version and not empty.

9.2 Important Instructions

1. Include the Javadoc tag `@author` in each class file.

2. **Don't change any function signature or class names.** The class names and function signature makes up the design contract, and any violation will be penalized by up to **50% reduction in score**.

9.3 Significant Dates

The homework is due on **Sept. 26th** at midnight. You can utilize a 3-day extension without any penalty or permission. The homework maybe marked as late, but won't be penalized up to **Sept. 29th** midnight. The final day to submit the homework would be **Sept. 30th**, and it would incur a **10% penalty**. No further extension will be allowed, so plan accordingly!

You can also accrue **10% bonus** points by submitting early. For each day you submitted early, limited to five days (from **Sept. 21st**), you get a 2% bonus point. These points are calculated based on your homework's graded score and not the total possible points.

10 Working Example

Below is a sample simulation scenario over three simulation cycles (the line numbers are not part of the simulation). In the first iteration, the user chooses option 1 to randomly generate a neighborhood grid, specify the grid size as 8, and to run the simulation over 3 months. In the second iteration, the simulation is carried out using an input file called "8 x 8.txt" for 5 months. In the third iteration, the user decides to exit the simulation.

```

1  Neighborhood Sports Preferences
2  =====
3  Keys: 1 (random grid); 2 (file input); 3 (exit)
4
5  Simulation Number 1: 1
6  Random grid
7  Enter grid width: 8
8  Enter the number of months: 3
9
10 Initial grid:
11
12 A4  E   A3  N   F0  N   A1  R2
13 E   E   S4  E   A1  E   S0  A3
14 E   F4  A3  S3  B4  B4  B4  E
15 N   R4  R4  A1  F3  N   N   S2
16 E   A0  N   R4  N   R3  E   N
17 B3  N   F2  S0  R2  B3  F0  R1
18 R0  F0  F4  S2  S4  N   R2  F0
19 E   B4  B0  R3  A3  N   R0  N
20
21 Updated grid for 1:
22
23 R0  B2  R0  E   A0  E   A2  A4
24 B2  B2  S5  B2  R0  B2  E   A4
25 E   A0  A4  E   B5  B5  S0  B2
26 E   A4  A4  A2  A0  E   E   E
27 E   R0  R2  A4  R2  R4  R0  E
28 E   E   F3  N   R3  E   F1  R2
29 F0  F1  F5  E   N   R2  R3  F1
30 B2  B5  B1  F0  R0  R2  R1  R2
31
32 Updated grid for 2:
33
34 F0  B3  F0  B2  R0  E   A3  A5
35 B3  B3  N   B3  F0  B3  B2  A5
36 E   A1  N   B2  N   N   E   E
37 E   A5  N   A3  A1  B2  B2  B2
38 E   A4  A4  A5  A4  A4  R1  B2
39 B2  F0  F4  R2  A4  R0  F2  R3
40 F1  F2  N   B2  R2  R3  R4  F2
41 B3  N   B2  F1  R1  R3  R2  R3
42
43 Updated grid for 3:

```

```

44
45 A0 B4 B0 B3 F0 B2 A4 N
46 B4 B4 B0 B4 B0 B4 B3 N
47 E A2 A0 B3 B0 B0 E E
48 E N A0 A4 A2 B3 B3 B3
49 B2 A5 A5 N N A5 R2 B3
50 E F1 F5 A4 A5 A4 F3 R4
51 F2 F3 N B3 R3 A4 R5 F3
52 E N B3 A0 R2 R4 R3 R4
53
54 Final grid:
55
56 A0 B4 B0 B3 F0 B2 A4 N
57 B4 B4 B0 B4 B0 B4 B3 N
58 E A2 A0 B3 B0 B0 E E
59 E N A0 A4 A2 B3 B3 B3
60 B2 A5 A5 N N A5 R2 B3
61 E F1 F5 A4 A5 A4 F3 R4
62 F2 F3 N B3 R3 A4 R5 F3
63 E N B3 A0 R2 R4 R3 R4
64
65 Keys: 1 (random grid); 2 (file input); 3 (exit)
66
67 Simulation Number 2: 2
68 Grid input from a file
69 File name: 8x8.txt
70 Enter the number of months: 5
71
72 Initial grid:
73
74 A0 B4 B0 B3 F0 B2 A4 N
75 B4 B4 B0 B4 B0 B4 B3 N
76 E A2 A0 B3 B0 B0 E E
77 E N A0 A4 A2 B3 B3 B3
78 B2 A5 A5 N N A5 R2 B3
79 E F1 F5 A4 A5 A4 F3 R4
80 F2 F3 N B3 R3 A4 R5 F3
81 E N B3 A0 R2 R4 R3 R4
82
83 Updated grid for 1:
84
85 R0 B5 B1 B4 B0 B3 R0 N
86 B5 E E E E E B4 E
87 E A3 A1 E E E E E
88 E A0 N A5 A3 B4 B4 B4
89 E N N A0 A0 N R3 B4
90 B2 F2 N A5 N A5 F4 R5
91 F3 F4 N B4 A4 A5 N F4
92 B2 E B4 R0 R3 A4 A4 R5
93
94 Updated grid for 2:
95
96 F0 N B2 B5 B1 B4 F0 E
97 N E E E E E B5 E
98 E A4 A2 E E E E E
99 E A1 A0 N A4 B5 B5 B5
100 E E A0 A1 N A0 R4 B5
101 E F3 N N A0 N F5 N
102 F4 F5 E B5 A5 N A0 F5
103 E B2 B5 R1 A4 A5 A5 N
104
105 Updated grid for 3:
106

```

```

107 A0 E B3 N B2 B5 A0 B2
108 E E E E E B2 N B2
109 E A5 A3 E E E E E
110 E A2 N A0 A5 N N N
111 B2 E A1 A2 A0 A1 F0 N
112 B2 F4 E A0 A1 A0 N N
113 F5 N E N N A0 A1 N
114 B2 B3 N F0 A5 N N N

```

115

116 Updated grid for 4:

117

```

118 R0 E E E B3 N R0 B3
119 E E E E E B3 B0 B3
120 E N A4 E E E E E
121 E A3 A0 N N A0 E E
122 B3 E A2 N N A2 F1 N
123 B3 F5 E A1 N N A0 N
124 F5 E E A0 A0 A1 A2 N
125 B3 B4 E F1 N A0 N N

```

126

127 Updated grid for 5:

128

```

129 F0 E E E B4 E F0 B4
130 E E E E E B4 B1 B4
131 E A0 A5 E E E E E
132 E A4 A1 A0 E A1 E E
133 B4 B2 A3 A0 A0 A3 F2 E
134 B4 F5 E A2 A0 A0 A1 N
135 F5 B2 B2 A1 A1 A2 A3 N
136 B4 B5 B2 F2 A0 A1 A0 N

```

137

138 Final grid:

139

```

140 F0 E E E B4 E F0 B4
141 E E E E E B4 B1 B4
142 E A0 A5 E E E E E
143 E A4 A1 A0 E A1 E E
144 B4 B2 A3 A0 A0 A3 F2 E
145 B4 F5 E A2 A0 A0 A1 N
146 F5 B2 B2 A1 A1 A2 A3 N
147 B4 B5 B2 F2 A0 A1 A0 N

```

148

149 Keys: 1 (random grid); 2 (file input); 3 (exit)

150

151 Simulation Number 3: 3

152

153 Process finished with exit code 0